

Fingertip Trajectory Recognition

Identify hand joints using MediaPipe's hand detection model. Once a specific gesture is recognized, the robot will initiate fingertip locking on the screen, track the fingertips, and generate their movement trajectory.

1. Program Logic

First, invoke the MediaPipe hand detection model to capture the camera image. Next, flip and process the image to extract hand information. Utilizing the connection lines between key points of the hand calculates the finger angles to determine the gesture.

Upon recognition of a specific gesture, the robot will proceed to identify and lock the fingertips on the screen, simultaneously tracing the movement trajectory of the fingertips on the display.

2. Operation Steps

Note: the input command should be case sensitive, and the keyword can be complemented by "Tab" key.

1) Start the robot, and enter the robot system desktop using VNC.

2) Click-on  to open the command-line terminal.

3) Run the command to disable app auto-start app service.

```
~/./stop_ros.sh
```

```
> ./stop_ros.sh
```

4) Enter the command to enable the depth camera service.

```
ros2 launch peripherals depth_camera.launch.py
```

```
> ros2 launch peripherals depth_camera.launch.py
```

5) Open a new terminal. Enter the command to navigate to the directory of the game.

```
cd ros2_ws/src/example/example/mediapipe_example
```

```
> cd ros2_ws/src/example/example/mediapipe_example
~/.ros2_ws/src/example/example/mediapipe_example 14:17:49
```

6) Type the command. Press “Enter” to run the program.

python3 hand_gesture.py

```
> cd ros2_ws/src/example/example/mediapipe_example
> python3 hand_gesture.py
INFO: Created TensorFlow Lite XNNPACK delegate for CPU.
[INFO] [1722608309.709830750] [hand_gesture]: start
```

7) If you want to exit the game, press “Ctrl+C” in the terminal interface.

After experiencing the game, you can enable the app service through commands or by restarting the robot. If the app is not enabled, the related app functions will not work. If the robot is restarted, the app will be automatically enabled.



Click  and enter the command. Press “Enter” to start the app, and wait for the buzzer to beep.

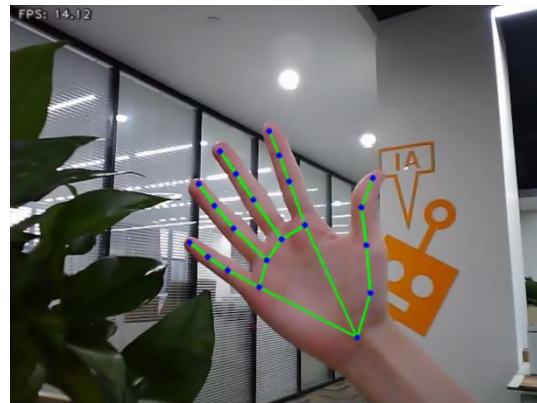
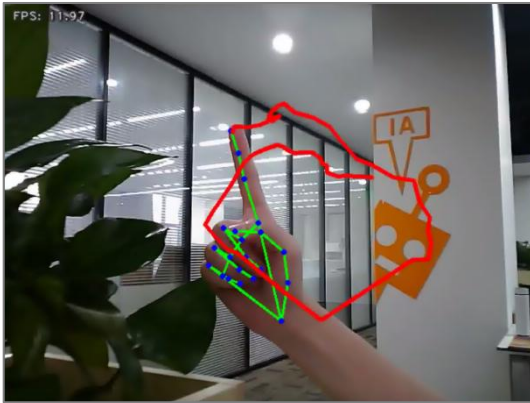
sudo systemctl restart start_node.service

```
LXTerminal
File Edit Tabs Help
> sudo systemctl restart start_node.service
```

3. Program Outcome

Once the game is enabled, position your hand within the camera's field of view. When your hand is recognized, the hand key points will be highlighted on the live camera feed.

If the robot detects the "1" gesture, the trajectory of your fingertip motion will begin to be recorded on the live camera feed. If it detects the "5" gesture, the recorded fingertip trajectory will be cleared.



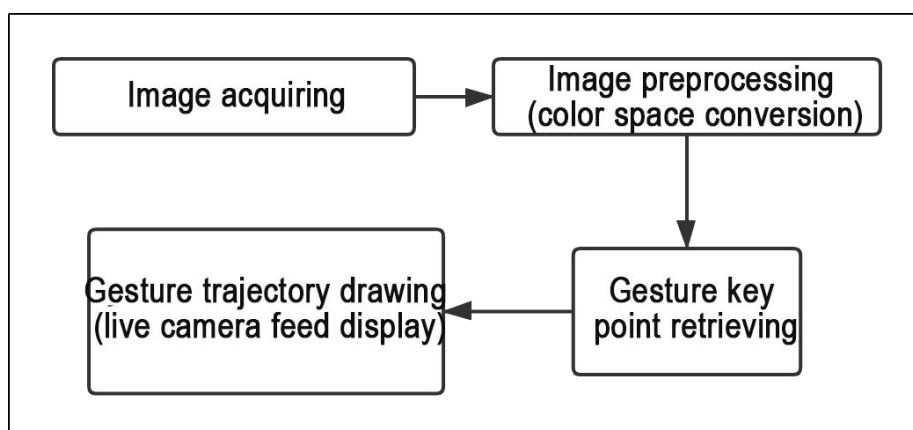
4. Program Analysis

The program file is saved in:

**`/home/ubuntu/ros2_ws/src/example/example/mediapipe_example/hand_
gesture.py`**

Note: Prior to making any alterations to the program, ensure to create a backup of the original factory program. Modify it only after creating the backup. Directly editing the source code file is prohibited to prevent inadvertent parameter modifications that could render the robot dysfunctional and irreparable!

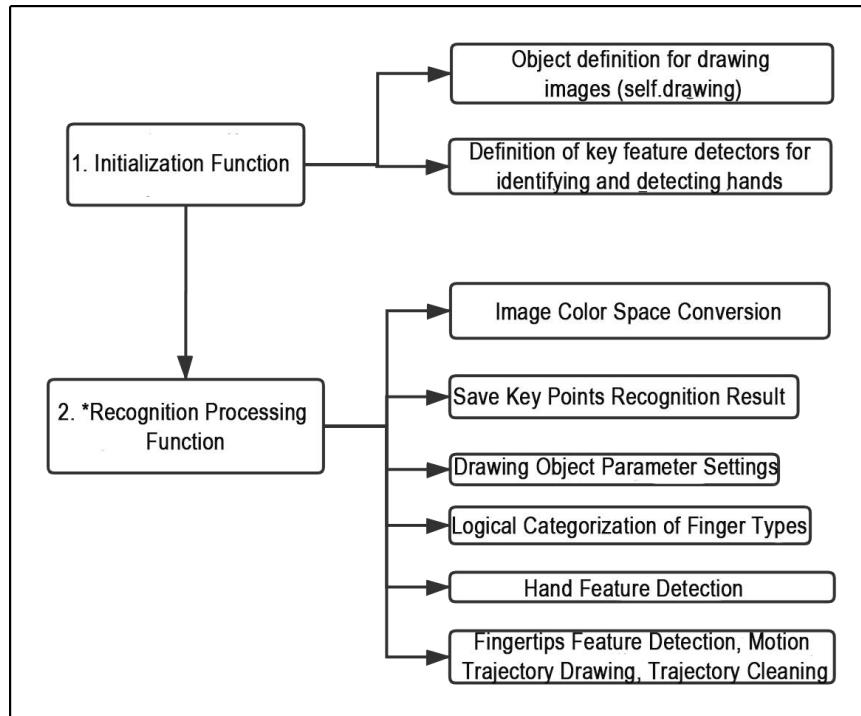
Based on the game's impact, the process logic of this game is organized as depicted in the figure below:



As depicted in the image above, the purpose of this game is to capture an image using the camera. Preprocess it by converting its color space for easier identification. Extract feature points corresponding to hand gestures from the

converted image. Determine different gestures (based on angles) through logical analysis of key feature points. Next, the trajectory of the recognized gesture is drawn on the live camera feed.

The program's logic flowchart extracted from the program files is illustrated in the figure below.



From the above diagram, it can be seen that the program's logical flow is mainly divided into initialization functions and recognition processing functions. The following document content will be written according to the program logic flow chart mentioned above.

- **Initialization function (init__(self, name))**

The program starts with the initialization function of the “HandTrajectoryNode” class, which defines some basic parameters. The important ones include the drawing object for drawing the hand landmarks and the hand detector for detecting the hand landmarks.

- **Define image drawing (self.drawing)**

```
121 | self.drawing = mp.solutions.drawing_utils
```

“self.drawing” is the drawing tool for the joint key points. It is used to connect the detected key points and visualize the hand trajectory. “mp” represents the mediapipe recognition module, which extracts the hand features. The “drawing_utils” package from mediapipe is used for drawing the hand landmarks.

- **Define detector for recognizing hand feature (self.hand_detector)**

```

123         self.hand_detector = mp.solutions.hands.Hands(
124             static_image_mode=False,
125             max_num_hands=1,
126             min_tracking_confidence=0.05,
127             min_detection_confidence=0.6
128         )
    
```

“self.hand_detector” is the hand feature detector. “mp” represents the mediapipe recognition module. “hands.Hands” is the hand feature extraction module. The following parameters are important:

static_image_mode	When the parameter is set to False, it will locate the maximum number of hands based on the value set by “max_num_hands”. By default, this parameter is set to False.
max_num_hands	Set the maximum number of hands to be detected, which is 1 by default.
min_detection_confidence	The minimum confidence threshold for the hand detection model ranges between 0 and 1. Only hands with confidence scores higher than this threshold will be detected. If the recognition performance is not optimal, you can adjust the threshold by ± 0.1 .

	However, it should not be set lower than 0.
min_tracking_confidence	This parameter is used for hand tracking and localization. It sets the minimum confidence threshold for the hand detection model to recognize and track a hand. If the confidence level of a detected hand falls below this threshold, it will be considered stationary and not tracked. If the recognition performance is not optimal, you can adjust the threshold by ± 0.1 . It should not be set lower than 0.

- **Recognition processing function (image_proc(self))**

After defining the basic tool parameters, the logical recognition process can be initiated. The following points include the main code analysis from the initial image preprocessing (color space conversion) to the final drawing of fingertip trajectories.

- **Image color space conversion**

```
158 image_flip = cv2.cvtColor(cv2.flip(image, 1), cv2.COLOR_BGR2RGB)
```

The input image needs to be converted from the BGR color space to the RGB color space to achieve accurate hand detection. The “cv2.cvtColor()” function is used for this purpose. “cv2.COLOR_RGB2BGR” is the color space conversion method. “bgr_image” is the output image. Keep the parameters as default.

- **Store keypoints recognition result**

```
161 results = self.hand_detector.process(image_flip)
```


The “process” function in “self.hand_detector” is responsible for extracting the key points of the hand. The results, which include the positions of the key points in the image, are stored in the “results” variable. It is used for subsequent logical processing.

- **Set drawing object parameter**

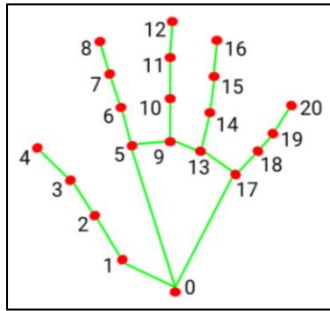
```
166 self.drawing.draw_landmarks(
167     result_image,
168     hand_landmarks,
169     mp.solutions.hands.HAND_CONNECTIONS)
```

Once the key feature points are detected, the drawing tool needs to be defined for drawing the hand landmarks and connections. The “bgr_image” parameter represents the input image; “hand_landmarks” represents the detected hand landmarks; “HAND_CONNECTIONS” defines the connections between the landmarks, based on the standard hand gesture feature point numbering.

- **Logical classification of finger types**

```
30 def hand_angle(landmarks):
31     """
32     计算各个手指的弯曲角度(calculate the bending angle of each finger)
33     :param landmarks: 手部关键点(the key points of the hand)
34     :return: 各个手指的角度(each finger's angle)
35     """
36     angle_list = []
37     # thumb 大拇指
38     angle_ = vector_2d_angle(landmarks[3] - landmarks[4], landmarks[0] - landmarks[2])
39     angle_list.append(angle_)
40     # index 食指
41     angle_ = vector_2d_angle(landmarks[0] - landmarks[6], landmarks[7] - landmarks[8])
42     angle_list.append(angle_)
43     # middle 中指
44     angle_ = vector_2d_angle(landmarks[0] - landmarks[10], landmarks[11] - landmarks[12])
45     angle_list.append(angle_)
46     # ring 无名指
47     angle_ = vector_2d_angle(landmarks[0] - landmarks[14], landmarks[15] - landmarks[16])
48     angle_list.append(angle_)
49     # pink 小拇指
50     angle_ = vector_2d_angle(landmarks[0] - landmarks[18], landmarks[19] - landmarks[20])
51     angle_list.append(angle_)
52     angle_list = [abs(a) for a in angle_list]
53     return angle_list
```

After extracting the hand points into the “results” variable, logical processing is required to identify the specific type of finger (e.g. thumb or index finger) by determining the angle relationships between the feature points. The “hand_angle” function is the input feature point set “landmarks(results)”. The “vector_2d_angle” function is used to calculate the angles between the corresponding feature points. The feature points corresponding to the elements in the “landmarks” set are shown in the following figure:



● Detect gesture feature

```

55 def h_gesture(angle_list):
56     """
57     通过二维特征确定手指所摆出的手势(Determine the gesture made by the fingers based on the two-dimensional features)
58     :param angle_list: 各个手指弯曲的角度(the angles of each finger's bending)
59     :return: 手势名称字符串(gesture name string)
60     """
61     thr_angle = 65.
62     thr_angle_thumb = 53.
63     thr_angle_s = 49.
64     gesture_str = "none"
65     if (angle_list[0] > thr_angle_thumb) and (angle_list[1] > thr_angle) and (angle_list[2] > thr_angle) and (
66         angle_list[3] > thr_angle) and (angle_list[4] > thr_angle):
67         gesture_str = "fist"
68     elif (angle_list[0] < thr_angle_s) and (angle_list[1] < thr_angle_s) and (angle_list[2] > thr_angle) and (
69         angle_list[3] > thr_angle) and (angle_list[4] > thr_angle):
70         gesture_str = "hand_heart"
71     elif (angle_list[0] < thr_angle_s) and (angle_list[1] < thr_angle_s) and (angle_list[2] > thr_angle) and (
72         angle_list[3] > thr_angle) and (angle_list[4] < thr_angle_s):
73         gesture_str = "nico-nico-qi"
74     elif (angle_list[0] < thr_angle_s) and (angle_list[1] > thr_angle) and (angle_list[2] > thr_angle) and (
75         angle_list[3] > thr_angle) and (angle_list[4] > thr_angle):
76         gesture_str = "hand_heart"
77     elif (angle_list[0] > 5) and (angle_list[1] < thr_angle_s) and (angle_list[2] > thr_angle) and (
78         angle_list[3] > thr_angle) and (angle_list[4] > thr_angle):
79         gesture_str = "one"
80     elif (angle_list[0] > thr_angle_thumb) and (angle_list[1] < thr_angle_s) and (angle_list[2] < thr_angle_s) and (
81         angle_list[3] > thr_angle) and (angle_list[4] > thr_angle):
82         gesture_str = "two"
83     elif (angle_list[0] > thr_angle_thumb) and (angle_list[1] < thr_angle_s) and (angle_list[2] < thr_angle_s) and (
84         angle_list[3] < thr_angle_s) and (angle_list[4] < thr_angle_s):
85         gesture_str = "three"
86     elif (angle_list[0] > thr_angle_thumb) and (angle_list[1] > thr_angle) and (angle_list[2] < thr_angle_s) and (
87         angle_list[3] < thr_angle_s) and (angle_list[4] < thr_angle_s):
88         gesture_str = "ok"
89     elif (angle_list[0] > thr_angle_thumb) and (angle_list[1] < thr_angle_s) and (angle_list[2] < thr_angle_s) and (
90         angle_list[3] < thr_angle_s) and (angle_list[4] < thr_angle_s):
91         gesture_str = "four"
92     elif (angle_list[0] < thr_angle_s) and (angle_list[1] < thr_angle_s) and (angle_list[2] < thr_angle_s) and (
93         angle_list[3] < thr_angle_s) and (angle_list[4] < thr_angle_s):
94         gesture_str = "five"
95     elif (angle_list[0] < thr_angle_s) and (angle_list[1] > thr_angle) and (angle_list[2] > thr_angle) and (
96         angle_list[3] > thr_angle) and (angle_list[4] < thr_angle_s):
97         gesture_str = "six"

```

After identifying the different finger types of the hand and determining their positions on the image, logical recognition processing of various gestures can be performed by implementing the “h_gesture” function.

In the “h_gesture” function depicted above, the parameters “thr_angle=65”, “thr_angle_thenum=53”, and “thr_angle_s=49” represent the angle threshold values for corresponding gesture logic points. These values have been empirically tested to ensure stable recognition effects. It is not recommended to alter them unless the logic processing effect is unsatisfactory, in which case adjustments within a range of ± 5 values are sufficient. The “angle_list[0,1,2,3,4]” corresponds to the five finger types associated with the palm.

Here's an example using the gesture "one":


```
77 elif (angle_list[0] > 5) and (angle_list[1] < thr_angle_s) and (angle_list[2] > thr_angle) and (  
78     angle_list[3] > thr_angle) and (angle_list[4] > thr_angle):  
79     gesture_str = "one"
```

The code presented represents the logical angle evaluation of the fingers for the “one” gesture. “angle_list[0]>5” checks whether the angle value of the thumb joint feature point in the image is greater than 5.

“angle_list[1]<thr_angle_s” checks if the angle feature of the index finger joint feature point is less than the predetermined value “thr_angle_s”. Similarly, “angle_list[2]<thr_angle” verifies if the angle feature of the middle finger feature point is less than the predetermined value “thr_angle”. The logical processing for the other two fingers, “angle_list[3]” and “angle_list[4]”, follows a similar method. When the above conditions are met, the current gesture feature is recognized as “one”, and the same principle applies to recognizing other gesture features.

Different gesture recognition involves distinct logical processing, but the overall logical framework remains similar. For recognizing other gesture features, refer to the previous paragraph.

● Detect fingertip feature, draw motion trajectory, and clear trajectory

The fingertip feature detection is shown below:

```
174 if self.state != State.TRACKING:  
175     if gesture == "one": # 检测食指手势, 开始指尖追踪(detect index finger gesture and start fingertip tracking)  
176         self.count += 1  
177         if self.count > 5:  
178             self.count = 0  
179             self.state = State.TRACKING  
180             self.points = []  
181     else:  
182         self.count = 0
```

The code above shows the logic judgement for when the gesture “one” is detected. The “self.count” is used for counting the number of frames that the current hand gesture has been held. If “self.count” is greater than 5, the gesture is confirmed as “one”. The duration of the gesture can be adjusted to get different recognition accuracies, usually set to around 5 to 7 frames. Once the gesture is confirmed as “one”, the current gesture state is set to “TRACKING”, indicating that you can move your hand. “self.points” and “points_list” are initialized to keep track of the position of the hand's adjacent points and the collection of feature points used to draw the trajectory line,

respectively.

The motion trajectory drawing is as below:

```
184 elif self.state == State.TRACKING:
185     if gesture != "two":
186         if len(self.points) > 0:
187             last_point = self.points[-1]
188             if distance(last_point, index_finger_tip) < 5:
189                 self.count += 1
190             else:
191                 self.count = 0
192                 self.points.append(index_finger_tip)
193         else:
194             self.points.append(index_finger_tip)
195     draw_points(result_image, self.points)
```

During the sliding process, the custom function “distance” calculates the distance between the previous and current points. A logical judgment is then made using “distance(last_point, index_finger_tip) < 5”. “5” is the distance in pixels between the two points. If you need to detect faster movements while maintaining accurate recognition, increase this value appropriately. It is recommended not to exceed 10. The movement feature points of the fingertip are stored, and “pixels” is an object used to store the fingertip information (x, y) for pixelization and drawing of the image.

The trajectory clearing process is shown below:

```
196 if gesture == "five":
197     self.state = State.NULL
198     self.points = []
199     draw_points(result_image, self.points)
```

When the gesture is recognized as “five”, the gesture recognition state is set to NULL (empty). The collection of current points, which is the position information of the trajectory points, will be cleared.